

Programming for Business

Session 17–18 (LO3)

NumPy & Pandas

FAST–NUCES

Miss Shumaila Arshad

Introduction to Data in Business

- Businesses generate data (sales, customers, finance).
- We use NumPy & Pandas to analyze it.

What is NumPy?

- NumPy is used for numerical computing and fast calculations using arrays.

NumPy Example 1

```
import numpy as np  
arr = np.array([1,2,3])  
print(arr)
```

Output:

```
[1 2 3]
```

- import → load library
- np → alias
- array → create array

NumPy Example 2

```
arr = np.array([1,2,3])  
print(arr + 2)
```

Output:

```
[3 4 5]
```

- Each element increases by 2

NumPy Example 3

```
arr = np.array([2,4,6])  
print(arr.mean())
```

Output:

4.0

- `mean()` → average

NumPy Real Life

- Sales forecasting, profit calculation, trend analysis

What is Pandas?

- Pandas is used for structured data (tables).

Pandas Example 1 – Series (1 dimensional array)

```
import pandas as pd  
s = pd.Series([10,20,30])  
print(s)
```

Output:

```
0 10  
1 20  
2 30
```

Pandas Example 2 - DataFrame

```
data = {'Name':['Ali','Sara'], 'Marks':[80,90]}  
df = pd.DataFrame(data)  
print(df)
```

Output:

Name	Marks
------	-------

Ali	80
-----	----

Sara	90
------	----

Filtering Example

```
print(df[df['Marks'] > 80])
```

Output:

Sara 90

Reading File

```
df = pd.read_csv('data.csv')  
print(df.head())
```

//head() is used to print first 5 rows from the file.

//tail() is used to print last 5 rows from the file.

Analysis

```
print(df.describe())
```

It gives a **summary of numerical data**. For example:

Marks

```
count    5.0  
mean    70.0  
std     10.0  
min     50.0  
25%     60.0  
50%     70.0  
75%     80.0  
max     90.0
```

It tells us:

- How data is distributed
- What are average values
- What is minimum & maximum

```
print(df['Marks'].mean()). //prints the average of marks
```

Combining NumPy & Pandas

- NumPy → calculations
- Pandas → structured data
- Used together in analytics

NumPy = **math engine**

Pandas = **table manager**

NumPy is used for numerical calculations, while Pandas is used to organize and analyze the data in tabular form.

Practice 1

```
arr = np.array([5,10,15])  
print(arr * 2)
```

Output:

```
[10 20 30]
```

Practice 2

```
data = {'A':[1,2], 'B':[3,4]}  
df = pd.DataFrame(data)  
print(df['A'])
```

Output:

1

2

Practice 3

Find students with marks > 80

Solution:

```
print(df[df['Marks'] > 80])
```

Practice question

```
import numpy as np
import pandas as pd
```

Step 1: Create NumPy array (sales data)

```
sales = np.array([100, 150, 200, 250, 300])
```

Step 2: Convert into Pandas DataFrame

```
df = pd.DataFrame({'Sales': sales})
```

Step 3: Calculate new column (10% increase)

```
df['Updated_Sales'] = df['Sales'] * 1.10
```

Step 4: Print DataFrame

```
print(df)
```

Step 5: Calculate average using NumPy

```
print("Average Sales:", np.mean(sales))
```

Output

	Sales	Updated_Sales
0	100	110.0
1	150	165.0
2	200	220.0
3	250	275.0
4	300	330.0

Average Sales: 200.0

Practice Question

```
import numpy as np
import pandas as pd
```

Step 1: Create NumPy arrays

```
cost = np.array([50, 60, 70, 80])
price = np.array([100, 120, 140, 160])
```

Step 2: Create DataFrame

```
df = pd.DataFrame({
    'Cost': cost,
    'Price': price
})
```

Step 3: Calculate Profit

```
df['Profit'] = df['Price'] - df['Cost']
```

Step 4: Filter high profit products

```
high_profit = df[df['Profit'] > 50]
```

Step 5: Print results

```
print(df)
print("\nHigh Profit Products:")
print(high_profit)
```

Step 6: Average profit using NumPy

```
print("\nAverage Profit:", np.mean(df['Profit']))
```

Output

	Cost	Price	Profit
0	50	100	50
1	60	120	60
2	70	140	70
3	80	160	80

High Profit Products:

	Cost	Price	Profit
1	60	120	60
2	70	140	70
3	80	160	80

Average Profit: 65.0

Practice Question

```
import pandas as pd
```

```
data = {'Name': ['Ali', 'Sara', 'Ahmed'], 'Marks': [80, 90, 70]}  
df = pd.DataFrame(data)
```

```
// Filter students with marks > 80
```

```
df = df[df['Marks'] > 80]
```

```
print("Filtered Data:")
```

```
print(df)
```

```
// Now print original data
```

```
print("\nOriginal Data:")
```

```
print(df)
```

Output

Filtered Data:

	Name	Marks
1	Sara	90

Original Data:

	Name	Marks
1	Sara	90

- **Reassigning df after filtering overwrites the original DataFrame, so only filtered data remains.**

Common Mistakes

- Forget import
- Wrong column name
- Syntax errors

Summary

- NumPy → fast calculations
- Pandas → data analysis
- Always understand code + output